Initial Project Plan

Dependencies

Client UI

- React
- Typescript

Platform Core

- MongoDB
- Django (Python with classes)

Milestones

- Having a game
- Create Account
- Log in
- View games list
- Chat
- Create game
- Join game
- Leaving a game
- View Dashboard
- Player Status
- Testing Milestones (defined later)

Detailed Project Task Breakdown

Project Overview

Platform: Gaming Platform with Client UI (React/TypeScript) and Platform Core (MongoDB/Django)

ALL SUB-TEAMS MUST COMPLETE

1. Sub-Team Class Diagram

Deliverable: class_diagram_[subteam].png or .svg

Tasks:

- ❑ Identify all core classes within your sub-team's area of responsibility
- ❑ Define attributes for each class
- ❑ Define methods for each class
- ❑ Map relationships between classes (inheritance, composition, aggregation, association)
- ❑ Identify and document interface points with other sub-teams' components
- ❑ Create visual diagram using UML notation
- ❑ Export as PNG or SVG format
- ❑ Review with team for completeness and accuracy

Requirements:

- Must show YOUR sub-team's classes only
- Must include clear interface points to other sub-teams
- Must use proper UML notation
- Format: PNG or SVG only

2. Use Case Diagram + Descriptions

Deliverable: use_cases_[subteam].pdf

Tasks:

- ❑ Identify 2-4 key use cases your sub-team is responsible for
- ❑ Create use case diagram showing:
    - All actors involved
    - System boundary
    - Use case interactions

    - Relationships (includes, extends, generalizations)
- ❑ Write detailed descriptions for EACH use case including:
    - **Actors:** Who interacts with this use case
    - **Preconditions:** What must be true before execution
    - **Main Flow:** Step-by-step normal execution path
    - **Postconditions:** System state after successful execution
    - **Exceptions:** Error conditions and alternative flows
- ❑ Format all content into a single PDF
- ❑ Review descriptions for clarity and completeness

Requirements:

- Minimum 2 use cases, maximum 4
- Each use case must have complete documentation
- Must show interaction with system
- Format: PDF only

3. State Machine Diagram

Deliverable: state_machine_[subteam].png or .svg

Tasks:

- ❑ Identify at least one important lifecycle/flow your sub-team manages
- ❑ Define all possible states
- ❑ Define all transitions between states
- ❑ Label transitions with triggers/events
- ❑ Identify initial and final states
- ❑ Document any guard conditions
- ❑ Create visual state machine diagram
- ❑ Export as PNG or SVG format
- ❑ Validate completeness of state coverage

Examples by Sub-Team:

- **Platform Core:** Lobby lifecycle, game session states
- **Client/UI:** User session states, screen navigation flow
- **Rules + Validation:** Validation flow, move processing states
- **Quality + Testing:** Test execution lifecycle
- **Integration + Release:** Build/deployment pipeline states

Requirements:

- Minimum one state machine diagram
- Must represent important lifecycle in your domain
- Format: PNG or SVG only

4. Design Presentation

Deliverable: Presentation in tutorial session

Tasks:

- ❑ Create presentation slides/materials
- ❑ Prepare content covering:
    - **The Problem:** What challenge your sub-team is addressing
    - **Design Solution:** Your proposed architecture/approach
    - **Integration Points:** How your design connects with other sub-teams
    - **Assumptions:** What you're assuming to be true
    - **Trade-offs:** Design decisions and their implications
- ❑ Practice presentation as a team
- ❑ Ensure ALL team members can answer questions about any part
- ❑ Prepare for Q&A from instructors and other teams
- ❑ Time the presentation appropriately

Requirements:

- Every team member must be prepared to present
- Every team member must be able to answer questions
- Must cover all required topics
- Be concise and clear

5. Planning Document

Deliverable: planning_[subteam].pdf

Tasks:

- ❑ List your sub-team's milestones for P2 (current phase)
- ❑ List your sub-team's milestones for P3 (implementation phase)
- ❑ Identify dependencies on other sub-teams:
    - What you need from Platform Core
    - What you need from Client/UI
    - What you need from Rules + Validation
    - What you need from Quality + Testing
    - What you need from Integration + Release
- ❑ Create high-level task breakdown for implementation phase
- ❑ Estimate effort/timeline for major tasks
- ❑ Identify risks and mitigation strategies
- ❑ Format into organized PDF document

Requirements:

- Must include P2 AND P3 milestones
- Must explicitly list inter-team dependencies
- Must provide implementation task breakdown
- Format: PDF only

SUB-TEAM SPECIFIC FOCUS AREAS

Platform Core Sub-Team

Primary Design Focus:

- ❑ Room/lobby lifecycle design
- ❑ Turn progression system
- ❑ Game state management architecture
- ❑ Core data models (User, Game, Room, etc.)
- ❑ Database schema design (MongoDB)
- ❑ Django backend API structure

Key Deliverables:

- Class diagram showing data models and core services
- Use cases: Create lobby, join game, progress turn, etc.
- State machine: Lobby lifecycle OR game session states
- Planning: Database setup, API endpoints, data persistence

Client/UI Sub-Team

Primary Design Focus:

- ❑ Screen flow design and navigation
- ❑ UI mockups for key screens
- ❑ Shared React components library
- ❑ User interaction design patterns
- ❑ State management approach (Redux, Context, etc.)
- ❑ Component hierarchy

Key Deliverables:

- Class diagram showing component structure and state management
- Use cases: Navigate screens, view dashboard, interact with UI, etc.
- State machine: User session states OR screen navigation flow
- Planning: Component development, styling approach, API integration

Additional Tasks:

- ❑ Create wireframes/mockups for major screens
- ❑ Define UI component naming conventions
- ❑ Establish styling/theming approach

Rules + Validation Sub-Team

Primary Design Focus:

- ❑ Move validation pipeline architecture
- ❑ Game-agnostic validation interfaces
- ❑ Error state handling and messaging
- ❑ Validation rule engine design
- ❑ Turn validation workflow
- ❑ Rule extensibility framework

Key Deliverables:

- Class diagram showing validation pipeline and rule interfaces
- Use cases: Validate move, handle invalid input, process turn, etc.
- State machine: Validation flow OR move processing states
- Planning: Validation framework, rule definitions, error handling

Additional Tasks:

- ❑ Define validation error taxonomy
- ❑ Design rule configuration format
- ❑ Plan for game-specific rule plugins

Milestones:
- **Initial alignment**
- move state created
- game state created
- error state created
- creation of service layer classes
- **basic end to end structure**
- define endpoints to accept moves
- define endpoints to validate and apply move
- define pipelines to send validation back to client
- define end point for user creation / login
- **playable game**
- full implementation of pipeline for move validation

- work with test subgroup to create proper test cases
- properly storing user stats

Dependencies:

- **Validation Dependencies:**
  - Win conditions, legal moves, turn order, tie rules, edge cases
  - Validation results (isValid, error, etc)

- **Authorization Dependencies:**
  - Which player is making the move
  - Which player belongs to which game room
  - Who's turn it is

- **Game Dependencies:**
  - What the state of a game is (Lobby, ready, in progress, finished)
  - Start conditions (minimum players, players ready)

- **Error Handling Dependencies:**
  - Validate move, turn, room, game status
  - Global exception handling

- **Storage Dependencies:**
  - Finished game results (win/ loss)

- **Testing Dependencies:**
  - Edge cases (illegal moves, boundary conditions, simultaneous requests, reconnection)

High-level task break down:

- create endpoint logic for user validation, creation, and deletion
  - endpoint logic for proper move validation
  - Proper handling of errors with games, user login, creation, deletion
  - Storage of finished game results
  - Pipeline to front end for game move validation
  - Room validation endpoint login

Quality + Testing Sub-Team

Primary Design Focus:

- ❑ Overall test strategy (unit, integration, E2E)
- ❑ Integration test architecture
- ❑ Demo script framework
- ❑ Test data management
- ❑ Continuous testing approach
- ❑ Quality metrics and reporting

Key Deliverables:

- Class diagram showing test framework and utilities
- Use cases: Run test suite, execute integration test, generate demo, etc.
- State machine: Test execution lifecycle
- Planning: Test coverage goals, testing tools, automation strategy

Additional Tasks:

- ❑ Define test coverage requirements per sub-team
- ❑ Select testing frameworks (Jest, Pytest, etc.)
- ❑ Design demo script structure
- ❑ Plan performance/load testing approach

Milestones:

1. Define test coverage requirements per sub-team
2. Finalize unit / integration / E2E strategy + ownership
3. Select and configure frameworks (Pytest)
4. Set up backend & frontend test structures (baseline coverage)
5. Implement unit tests (backend + frontend)
6. Implement API integration tests
7. Implement frontend–backend flow tests
8. Implement core gameplay E2E scenarios
9. Design demo script framework + test data strategy
10. Build integration test architecture

Dependencies:

- Stable API contracts and endpoints from all sub-teams
- Game state and validation pipeline finalized
- Feature-complete staging build for demo script execution

High-level Task Breakdown:

- Define unit, integration, and E2E test strategy and ownership per sub-team
- Select and configure testing frameworks (Jest, Pytest, Playwright/Cypress)
- Design test framework class diagram with utilities and data factories
- Build state machine for test execution lifecycle (queued → running → passed/failed → reported)
- Design test data seeding, isolation, and teardown patterns
- Build demo script structure with repeatable seed, run, and teardown flow

Integration + Release Sub-Team

Primary Design Focus:

- ❑ GitLab repository setup and structure
- ❑ Branching strategy (GitFlow, trunk-based, etc.)
- ❑ Build/run instructions and automation
- ❑ README documentation structure
- ❑ CI/CD pipeline design
- ❑ Deployment workflow
- ❑ Environment management (dev, staging, prod)

Key Deliverables:

- Class diagram showing build/deployment system components
- Use cases: Build project, deploy to environment, merge code, etc.
- State machine: Build/deployment pipeline states
- Planning: Repo structure, CI/CD tools, documentation standards

Additional Tasks:

- ❑ Create comprehensive README template
- ❑ Set up GitLab project structure
- ❑ Configure CI/CD pipelines
- ❑ Define merge request process
- ❑ Establish code review standards
- ❑ Document local development setup

Milestones:

- Setup and maintain project README file with instructions on how to build the project and use it.
- Get started with CI/CD pipeline and integrated automated tests to ensure continuous, smooth and bug free integration
- Setup build scripts
- Merge changes from different subteams seamlessly

Dependencies

- Test scripts from the quality and testing sub team which will get integrated into the CI pipeline
- Commits from all sub teams
- Release versioning rules

High level task breakdown

- Define git strategy (trunk based vs gitflow)
- Configure merge request approval requirements
- Establish commit message conventions
- Upkeep the README
- Create branch protection rules

SUBMISSION CHECKLIST

Required Files Per Sub-Team

- ❑ class_diagram_[subteam].png or .svg
- ❑ use_cases_[subteam].pdf
- ❑ state_machine_[subteam].png or .svg
- ❑ planning_[subteam].pdf

Submission Process

- ❑ Verify all files are named correctly
- ❑ Verify all files are in correct format
- ❑ Submit all files to D2L before deadline
- ❑ Confirm submission was successful

INDIVIDUAL REQUIREMENTS

Every Team Member Must:

1. Worklog on Trello

- ❑ Log minimum 3 hours per week
- ❑ **CRITICAL:** Do NOT log all hours on due date
- ❑ Distribute work entries throughout the week
- ❑ Include specific task descriptions
- ❑ Be honest about time spent

Trello Worklog Format:

- Task name
- Time spent
- Date completed
- Brief description of work done

2. Peer Evaluation

- ❑ Complete peer evaluation via ITP Metrics
- ❑ Submit before deadline
- ❑ Provide constructive feedback
- ❑ Be honest and fair in assessments

3. Presentation Preparedness

- ❑ Understand all aspects of your sub-team's design
- ❑ Be able to explain your personal contributions
- ❑ Be able to answer questions about:
    - Your class diagram
    - Your use cases
    - Your state machine
    - Your planning document
    - Integration with other sub-teams

KEY REMINDERS & PRINCIPLES

Platform-First Thinking

- ❑ Focus on shared platform systems
- ❑ Design for reusability across different games
- ❑ Don't just think about specific game logic
- ❑ Build abstractions that work for multiple scenarios

Consistency Across Sub-Teams

- ❑ Coordinate naming conventions
- ❑ Align on interface definitions
- ❑ Use consistent diagram notation
- ❑ Cross-validate designs with dependent teams

Stubs & Integration Points

- ❑ Identify where database connections will reside
- ❑ Identify where server/API connections will be made
- ❑ Define clear interface contracts
- ❑ Document stub implementations needed for P2

Communication Requirements

- ❑ Regular sync meetings with dependent sub-teams
- ❑ Document integration point agreements
- ❑ Escalate conflicts/blockers early
- ❑ Share design updates proactively

PROJECT MILESTONES

Overall Project Milestones (All Teams)

- ❑ Having a game (end-to-end functionality)
- ❑ Create Account
- ❑ Log in
- ❑ View games list
- ❑ Chat functionality
- ❑ Join game
- ❑ View Dashboard
- ❑ Player Status
- ❑ Testing Milestones (to be defined by Quality team)

P2 Phase (Current - Design)

- ❑ Complete all design deliverables
- ❑ Present design in tutorial
- ❑ Submit all required documents
- ❑ Validate design with instructors

P3 Phase (Implementation)

- Implement designed systems
- Integration testing
- End-to-end testing
- Final demonstration
- Project documentation

DEPENDENCIES MAP TEMPLATE

Use this template in your planning document:

What We Need From Other Teams:

From Platform Core:

- API endpoint: [description]
- Data model: [description]
- Service: [description]

From Client/UI:

- Component: [description]
- Screen: [description]
- State management: [description]

From Rules + Validation:

- Validation interface: [description]
- Rule definition: [description]
- Error handling: [description]

From Quality + Testing:

- Test utilities: [description]
- Mock data: [description]
- Testing guidance: [description]

From Integration + Release:

- CI/CD pipeline: [description]
- Build scripts: [description]
- Documentation standards: [description]

What We Provide To Other Teams:

To Platform Core:

- [Description of what you're providing]

To Client/UI:

- [Description of what you're providing]

To Rules + Validation:

- [Description of what you're providing]

To Quality + Testing:

- [Description of what you're providing]

To Integration + Release:

- CI/CD piplines
- Automated build scripts
- Documentation and README updates

TIMELINE RECOMMENDATIONS

Week 1

Initial sub-team meeting

- Assign roles and responsibilities
- Begin class diagram
- Identify use cases
- Start Trello logging

Week 2

- Complete class diagram draft
- Complete use case descriptions
- Begin state machine diagram

- Coordinate with dependent teams
- Continue Trello logging

Week 3

- Finalize all diagrams
- Complete planning document
- Prepare presentation
- Review all deliverables
- Practice presentation
- Continue Trello logging

Week 4 (Submission Week)

- Final review of all documents
- Submit to D2L
- Deliver presentation
- Complete peer evaluation
- Final Trello updates

QUALITY CHECKLIST

Before submitting, verify:

Class Diagram

- All classes have attributes and methods
- Relationships are clearly labeled
- Interface points to other teams are marked
- UML notation is correct
- Diagram is readable and well-organized

Use Case Diagram + Descriptions

- 2-4 use cases identified
- All actors shown

- System boundary clear
- Each use case has complete description
- All sections (actors, preconditions, main flow, postconditions, exceptions) included

State Machine Diagram

- Initial and final states marked
- All transitions labeled
- All states reachable
- No orphaned states
- Diagram represents actual system flow

Planning Document

- P2 milestones listed
- P3 milestones listed
- Dependencies clearly identified
- Task breakdown is realistic
- Document is well-organized

Presentation

- Covers all required topics
- All team members prepared
- Timing is appropriate
- Visuals are clear
- Q&A preparation complete

TOOLS & RESOURCES

Recommended Diagramming Tools

- **UML Diagrams:** Lucidchart, Draw.io, PlantUML, StarUML
- **State Machines:** Draw.io, Lucidchart, Visual Paradigm
- **Mockups (UI team):** Figma, Sketch, Adobe XD

Document Creation

- **PDF Generation:** Microsoft Word, Google Docs, LaTeX
- **Diagrams:** Export from diagramming tools

Collaboration

- **Trello:** Worklog tracking
- **GitLab:** Code repository (Integration team to set up)
- **Shared Documents:** Google Drive, OneDrive

COMMON PITFALLS TO AVOID

- Don't wait until the due date to log Trello hours
- Don't work in isolation - communicate with other teams
- Don't create overly game-specific designs
- Don't forget to show integration points in diagrams
- Don't submit incorrect file formats
- Don't skip practice for the presentation
- Don't assume other teams know what you need - document it!

SUCCESS CRITERIA

Your sub-team's design is successful if:

- All deliverables are complete and submitted on time
- Designs are coherent and well-thought-out
- Integration points with other teams are clear
- All team members understand the design
- Presentation demonstrates deep understanding
- Individual worklogs show consistent effort
- Dependencies are identified and communicated
- Design is platform-first and reusable

Document Version: 1.0

Last Updated: [Current Date]

Sub-Team: _____